Panduan Pelaksanaan Pengujian Bizgrow (Bab III–IV)

Panduan ini menjawab: **pengujian pakai apa, dijalankan bagaimana, dan bukti apa yang disimpan** supaya setiap tabel Bab IV (4.6–4.12) bisa diisi dan dipertanggungjawabkan. Semua skrip ada di `web/scripts/pengujian/`. Lembar kerjanya adalah `Lembar_Pengujian_Bizgrow.xlsx` (versi dengan koreksi faktual: `Lembar_Pengujian_Bizgrow_revisi.xlsx`, lihat `TEMUAN_DAN_REVISI.md`).

> **Prinsip:** skrip hanya *mengumpulkan* data dan bukti. Angka di skripsi harus berasal dari **eksekusi Anda sendiri** di lingkungan uji, bukan dari saya. Hasil percobaan saya di komputer lokal (server tiruan, tanpa Groq) hanya dipakai untuk memastikan skrip berjalan dan **tidak boleh dikutip** di skripsi.

---

1. Alat per jenis pengujian

Tabel	Jenis uji	Alat	Cara kerja singkat	Bukti yang tersimpan
4.6 Fungsional (24 skenario)	Black-box	**Manual lewat UI/aplikasi** + perekam layar	Jalankan langkah di sheet `Fungsional`, bandingkan dengan hasil yang diharapkan	Tangkapan layar tiap skenario (`POS-01.png`, …)
4.7 Keamanan (7 kontrol)	Black-box keamanan	`keamanan.mjs`, `ratelimit.mjs` (Node) + manual (skenario 3)	Mengirim permintaan HTTP nyata ke server uji, mencatat status & isi respons	`bukti/<tanggal>/keamanan/*.txt` + `ringkasan.csv`
4.8 Performa realtime	Pengukuran waktu	`performa.mjs pos` (Socket.io client)	Checkout POS lewat API → ukur sampai event `pos-transaction` diterima	`performa/pos.csv`, `pos-statistik.json`
4.8 Waktu respons AI	Pengukuran waktu	Diambil otomatis dari `ai.mjs`	Waktu per panggilan	kolom *Waktu (ms)* di sheet AI
4.9–4.10 Evaluasi AI (70 kasus × 3)	Evaluasi keluaran	`ai.mjs` (+ `acuan-analitis.sql`)	Memanggil endpoint AI, menyimpan keluaran mentah, memberi *usulan* nilai	`ai/mentah-*.jsonl`, `ai/*.csv`
4.11 Laporan vs data	Pemeriksaan	Query SQL + halaman laporan	Bandingkan angka laporan dengan SELECT pada periode sama	Tangkapan layar laporan + hasil query
4.12–4.13 Kuesioner	Survei	Lembar Likert (sheet `Kuesioner`)	Diisi pemilik setelah uji coba	Lembar kuesioner bertanda tangan/foto

Alasan memilih skrip Node untuk keamanan/AI/performa: permintaan HTTP persis sama dengan yang dikirim aplikasi, hasilnya terukur (milidetik), dapat diulang, dan menyimpan keluaran mentah sehingga dapat diperiksa penguji/pembimbing.

2. Persiapan (sekali)

1. **Lingkungan uji.** Server yang dipakai harus punya Redis (Upstash) agar pembatasan login bekerja; bila Redis mati, sistem *fail-open* dan skenario 4 pasti gagal. Catat URL di sheet `Petunjuk` B10. Tenant uji dibuat sebagai akun baru, terpisah dari akun asli kedua UMKM (data sintetis).
2. **Data uji** sesuai sheet `Data Uji`: Tenant A (Esteh Uji, CAFE) dan Tenant B (Bubur Uji, RESTORAN), produk, stok, COA. Kode akun COA di sheet harus sama persis dengan yang ada di Bizgrow (jika beda, ubah acuan *sebelum* uji dan catat). Isi transaksi sintetis Tenant A ≥ 3 bulan kalender, ≥ 20 transaksi di bulan berjalan (butuh untuk kasus A04–A11).
3. **Peran.** Kedua studi kasus tidak punya karyawan, jadi **seluruh uji POS/fungsional dijalankan dengan akun pemilik** (web dan mobile). Satu-satunya yang butuh peran lain adalah **skenario keamanan 3 (otorisasi)**: buat satu karyawan *kasir* uji di Tenant A (login lewat `/portal/<login_slug>` dengan PIN). Itu menguji kontrol

sistem, bukan kebutuhan UMKM; jika dosen setuju, skenario ini dapat dihapus dari Tabel 3.2/4.7 (jumlah skenario menjadi enam).

4. **Konfigurasi skrip:**

```
cd web/scripts/pengujian
cp config.contoh.json config.json     # isi URL, email/sandi pemilik A & B, unitId, slug, produk performa
```

   Isi juga `socketUrl` = alamat VPS Socket.io (nilai `PUBLIC_SOCKET_URL` di Vercel); skrip performa memakai `/api/socket-token`, jalur yang sama dengan dasbor web. `config.json` dan `bukti/` otomatis diabaikan git (berisi kata sandi). Node ≥ 18, `npm install` di folder `web/` sudah cukup.

5. **Catat konfigurasi** di sheet `Petunjuk`: tanggal, nilai `GROQ_MODEL` (per 8 Nov 2026 tidak diset di Vercel), commit yang ter-deploy (`git rev-parse --short HEAD`), perangkat & jaringan.
6. Opsional: `export SESI_BUKTI=uji-final` agar semua skrip menulis ke `bukti/uji-final/` (default: folder bertanggal hari itu).

3. Urutan eksekusi dan cara mengisi tiap sheet

Langkah 1 — Keamanan (Tabel 4.7)

```
node keamanan.mjs            # skenario 1,2,3,5,6,7  (≈ 1 menit)
# tunggu ≥ 15 menit tanpa login dari IP yang sama
node ratelimit.mjs           # skenario 4a (11 percobaan web) & 4b (16 percobaan API)
```

- Skenario **3 (kasir)**: login portal staf sebagai kasir di browser, salin cookie `staff_session_token` dari DevTools → isi `kasirA.staffSessionToken` di `config.json`. Atau uji manual dan foto layar penolakannya.
- Skenario **5**: skrip memeriksa respons API; **bukti hash** harus Anda ambil sendiri: `SELECT LEFT(password,10) FROM users WHERE email='…';` → harus `$argon2id$`. Foto hasil query.
- Skenario **6a/7a (halaman web)**: skrip mencoba lewat cookie; tambahkan juga tangkapan layar browser sebagai bukti visual.
- ⚠️ `ratelimit.mjs` mengunci IP/akun ±15 menit. Jalankan **paling akhir**; jangan login dari IP itu sesudahnya.
- Status di `ringkasan.csv` adalah **usulan**. Anda memutuskan Lulus/Gagal; kolom *Bukti* diisi nama berkas (`keamanan/6b_api_tenant_B.txt`).

Langkah 2 — Fungsional (Tabel 4.6) dan Performa (Tabel 4.8)

- **Fungsional**: manual, 24 skenario. Untuk tiap skenario ambil tangkapan layar (kondisi awal → hasil). Beberapa titik penting:
  - `POS-04` (bayar kurang): peringatan "Uang belum cukup" ada di UI; API tidak memvalidasi — uji lewat UI dan catat.
  - `STK-06` (jual melebihi stok): aturan Bizgrow = **ditolak 409** "Stok … tidak mencukupi! (Sisa: n)".
  - `POS-07` (mobile → web) dan `DSB-05` (dasbor realtime): gunakan dua perangkat, rekam layar.
- **Performa POS**:

```
node performa.mjs pos        # 20 percobaan, jeda 3 dtk → performa/pos.csv (butuh socketUrl di config.json)
```

  Mengukur dari permintaan checkout sampai event Socket.io diterima, di **satu mesin satu jam** (tanpa selisih jam). Ini berbeda dari definisi "HP → laptop" di sheet; jika ingin kondisi itu, rekam layar 60 fps pada kedua perangkat dan hitung selisih frame, lalu tulis di *Catatan percobaan*. Tulis kondisi uji (Wi-Fi, perangkat, lokasi server) di sheet `Performa` A27–A30.
- **Peringatan stok realtime**: fitur ini tidak ada (lihat `TEMUAN_DAN_REVISI.md` §B). Putuskan hapus atau ganti dengan `node performa.mjs stok` (waktu respons `/api/low-stock`) dan beri label yang jujur di Tabel 4.8.

Langkah 3 — Acuan analitis lalu evaluasi AI (Tabel 4.9–4.10)

1. **Pada hari uji, sebelum chat dijalankan**, jalankan `acuan-analitis.sql` (ganti `@unit`), foto hasilnya, lalu salin `acuan-analitis.contoh.json` → `acuan-analitis.json` dan isi angkanya. Setelah itu **acuan dikunci**.

2. Jalankan tiap jenis, tiga kali (default `--run 1,2,3`), jeda 4 dtk (batas kuota AI: 20/menit & 300/hari per akun):
   ```
   node ai.mjs --jenis ekstraksi
   node ai.mjs --jenis akun
   node ai.mjs --jenis analitis
   ```
   Kasus yang kena galat server/kuota ditandai `ULANGI` dan **tidak** dihitung sebagai kesalahan model — jalankan ulang dengan `--kasus E07,P03 --run 2`.
3. Penilaian: ekstraksi & akun diberi *usulan* deterministik (kategori/produk/qty/nominal; akun & kas). Analitis: skrip hanya memeriksa apakah angka acuan muncul; **narasi dinilai manual** sesuai rubrik Bab III.
4. Penilai kedua: 14 kasus bertanda "Ya" (kolom *Dinilai ulang?*) dinilai ulang tanpa melihat nilai Anda; isi `Nilai penilai 2`.
5. Salin ke lembar (tidak menimpa berkas asli):
   ```
   python3 isi_lembar.py --lembar Lembar_Pengujian_Bizgrow_revisi.xlsx --bukti bukti/<tanggal> --isi-nilai
   ```
   Tanpa `--isi-nilai` hanya keluaran ringkas & waktu yang diisi. Periksa tiap sel; rumus `Rekap Bab IV` tetap utuh.

Jalur yang dievaluasi untuk ekstraksi & akun adalah `POST /api/app/ai-entry` (temperature 0, max_tokens 300; nominal dihitung model). Jalur web `entry` berbeda: nominal produk dihitung **server** dari harga × qty. Tulis jalur yang diuji di Bab III/IV (lihat `TEMUAN_DAN_REVISI.md` §A5). Pratinjau chat ("beli gula 50rb") memakai **parser regex**, bukan model, sehingga bukan objek evaluasi AI.

Langkah 4 — Laporan vs data (Tabel 4.11)

Untuk periode yang sama, bandingkan angka halaman Laporan/Dasbor dengan query (`transaksi`, `products`) — templatenya ada di `acuan-analitis.sql`. Isi sheet `Laporan` (nilai laporan, nilai query, selisih otomatis). Gunakan **data nyata** kedua UMKM hanya dengan persetujuan pemilik (Bab III e); jika tidak, gunakan tenant uji.

Langkah 5 — Kuesioner (Tabel 4.12–4.13)

Setelah masa uji coba berakhir; satu pemilik per UMKM (n = 1). Foto/arsipkan lembar yang diisi. Butir layanan "tidak mengalami" dicatat TM.

4. Paket bukti untuk lampiran skripsi

```
bukti/<tanggal>/
  keamanan/  1a_… 6b_… 7_cek_B_sesudah.txt  ringkasan.csv  ringkasan-ratelimit.csv
  ai/        mentah-*.jsonl  ekstraksi-*.csv  akun-*.csv  analitis-*.csv
  performa/  pos.csv  pos-statistik.json
  tangkapan-layar/  POS-01.png … DSB-05.png, hash-argon2.png, acuan-sql.png
```

Token dan cookie disamarkan otomatis di berkas `.txt`. Jangan melampirkan `config.json`.

5. Yang sengaja tidak diklaim

- Keamanan: "kontrol berfungsi pada skenario yang diuji", bukan "bebas kerentanan" (sesuai Bab III).
- Performa: hanya berlaku untuk kondisi uji yang ditulis; median & rentang, bukan rata-rata.
- AI: "akurasi" = proporsi keluaran *sesuai penuh* pada 70 kasus, untuk versi model/prompt/commit yang dicatat.